LAPORAN TUGAS 2

Membuat Block Cipher “Baru”

IF4020 Kriptografi

Semester I Tahun Akademik 2026/2027



Disusun oleh

Zulfaqqar Nayaka Athadiansyah 13523094
Azfa Radhiyya Hakim 13523115
Brian A. Hadian 13523048

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

Pernyataan Integritas Akademik

Tugas dan laporan ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Zulfaqqar Nayaka Athadiansyah (13523094)



Azfa Radhiyya Hakim (13523115)



Brian A. Hadian (13523048)

Pembagian Tugas

Anggota	**Kontribusi**
13523094	Laporan, optimisasi, analisis keamanan
13523115	CLI, pengujian independen„ dan integrasi
13523048	Rancangan dan implementasi core block cipher

Daftar Isi

Pernyataan Integritas Akademik

Pembagian Tugas

1 Pendahuluan **1**

1.1 Latar Belakang . . . 1

1.2 Block Cipher Kontemporer . . . 1

1.3 Gagasan Utama . . . 2

2 Dasar Teori **3**

2.1 *Confusion* dan *Diffusion* . . . 3

2.2 Jaringan Feistel . . . 3

2.3 Substitusi dan Permutasi . . . 4

2.4 Operasi Tambahan . . . 5

2.5 Fungsi Kompresi SHA-256 . . . 5

2.6 Penjadwalan Kunci . . . 5

2.7 Mode Operasi dan *Padding* . . . 5

3 Perancangan Block Cipher **6**

3.1 Desain Blok dan Kunci . . . 6

3.2 Struktur Putaran dan Enkripsi . . . 6

3.3 Komponen Algoritma . . . 8

3.3.1 S-box . . . 8

3.3.2 Kompresi Cabang Dalam . . . 8

3.3.3 Difusi Global dan Permutasi Bit . . . 9

3.4 Penjadwalan Kunci . . . 9

3.5 Dekripsi . . . 11

4 Implementasi dan Simulasi **12**

4.1 Struktur . . . 12

4.2 Antarmuka Program . . . 12

4.3 Hasil Uji Coba . . . 13

4.4 Verifikasi Integritas . . . 13

4.5 Analisis 14

5 Analisis Keamanan **15**

5.1 *Avalanche Effect* 15

5.2 Entropi 16

5.3 Histogram 16

5.4 Keterbatasan Analisis 18

6 Penutup **19**

6.1 Kesimpulan 19

6.2 Saran 19

6.3 Lampiran 19

1 Pendahuluan

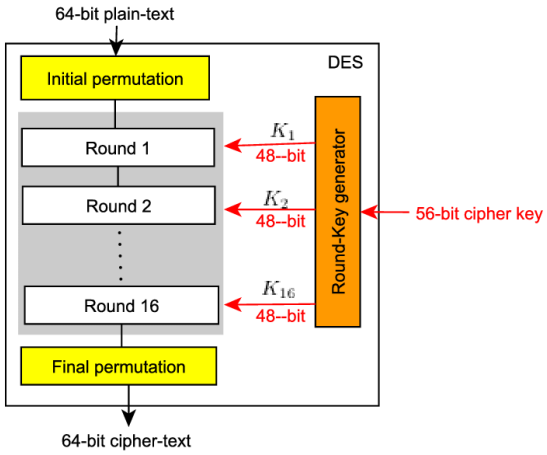
1.1 Latar Belakang

Block cipher merupakan algoritma enkripsi simetris yang mengamankan data dengan membagi pesan ke dalam blok-blok bit berukuran tetap dan mengolahnya melalui beberapa putaran transformasi. Untuk menjamin keamanannya, rancangan algoritma harus memenuhi prinsip confusion dan diffusion menggunakan operasi substitusi serta transposisi. Pada tugas ini, dilakukan perancangan dan implementasi algoritma block cipher baru beserta pengujian performa keamanannya yang meliputi avalanche effect, entropi, dan distribusi histogram dalam berbagai mode operasi enkripsi.

1.2 Block Cipher Kontemporer

Ada beberapa algoritma block cipher yang telah dikembangkan dan digunakan secara luas dan beberapa di antaranya sudah dibahas dalam kuliah IF4020 Kriptografi. Beberapa contoh algoritma *block cipher* yang populer ini adalah DES, 3DES, AES, dan Blowfish.

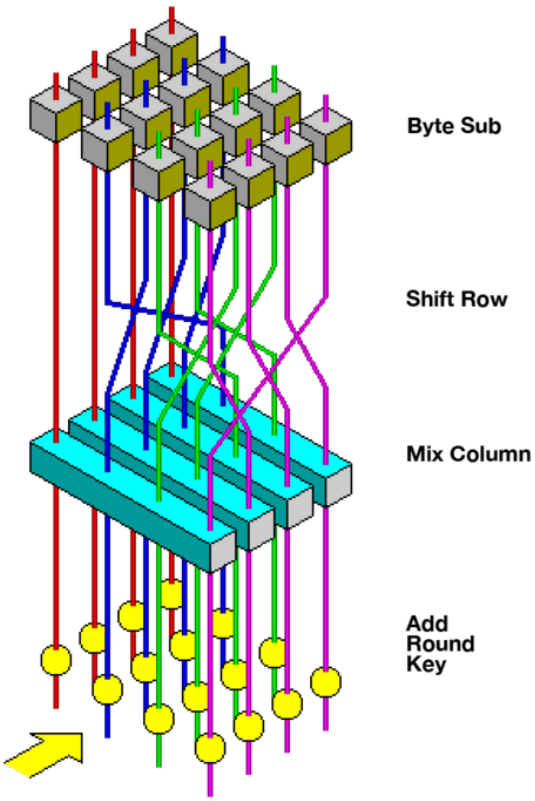
DES (Data Encryption Standard) merupakan algoritma block cipher yang dikembangkan oleh IBM pada tahun 1970-an dan diadopsi sebagai standar enkripsi oleh pemerintah Amerika Serikat pada tahun 1977. DES menggunakan blok berukuran 64-bit dan kunci berukuran 56-bit. Meskipun DES pernah menjadi standar enkripsi yang populer, algoritma ini kini dianggap tidak aman karena panjang kunci yang relatif pendek dan kerentanannya terhadap serangan brute-force. Untuk mengatasi kelemahan ini, 3DES (Triple DES) dikembangkan dengan menerapkan algoritma DES tiga kali secara berurutan, sehingga meningkatkan keamanan dengan panjang kunci efektif 112-bit atau 168-bit. Namun, 3DES juga dianggap lambat dan tidak efisien dibandingkan algoritma modern lainnya.



Gambar 1: Struktur DES

Adapun AES (Advanced Encryption Standard) adalah algoritma block cipher yang dikembangkan untuk menggantikan DES dan 3DES. AES menggunakan panjang kunci yang fleksibel yakni 128-bit, 192-bit, atau 256-bit, dan blok berukuran 128-bit. AES dikenal karena keama-

nannya yang tinggi dan efisiensinya dalam implementasi perangkat keras maupun perangkat lunak.



Gambar 2: Struktur AES

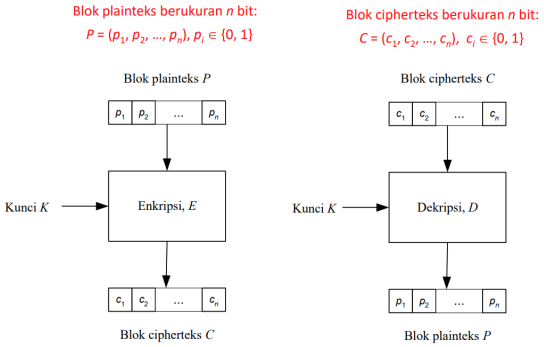
Terakhir, Blowfish adalah algoritma block cipher yang dirancang oleh Bruce Schneier pada tahun 1993. Blowfish menggunakan panjang kunci variabel hingga 448-bit dan blok berukuran 64-bit. Algoritma ini dikenal karena kecepatannya dan keamanan yang baik, meskipun kini telah digantikan oleh algoritma yang lebih modern seperti AES.

1.3 Gagasan Utama

Ide utama OATSIDE adalah menggabungkan dua pendekatan klasik, yaitu **jaringan Feistel** seperti pada DES dan lapisan **substitusi-permutasi** seperti pada AES, ke dalam satu putaran. Kami memecah blok menjadi enam cabang, empat di antaranya diproses dengan Feistel dan dua sisanya diproses dengan substitusi-permutasi. Kami juga menambahkan mekanisme seperti kompresi dan difusi global karena cabang dalam dan luar belum saling memengaruhi dalam satu putaran. Akibatnya, perubahan satu bit pada blok input tidak akan menyebar ke seluruh blok setelah satu putaran. Mekanisme ini juga menambah kompleksitas dan keamanan algoritma.

2 Dasar Teori

Block cipher merupakan algoritma enkripsi simetris yang mengamankan data dengan membagi pesan ke dalam blok-blok bit berukuran tetap (biasanya 64 bit, 128 bit, dsb) dan mengolahnya melalui beberapa putaran transformasi. Untuk menjamin keamanannya, rancangan algoritma harus memenuhi prinsip *confusion* dan *diffusion* dengan tetap menggunakan operasi yang sederhana seperti substitusi dan transposisi.



Gambar 3: Ilustrasi Block Cipher

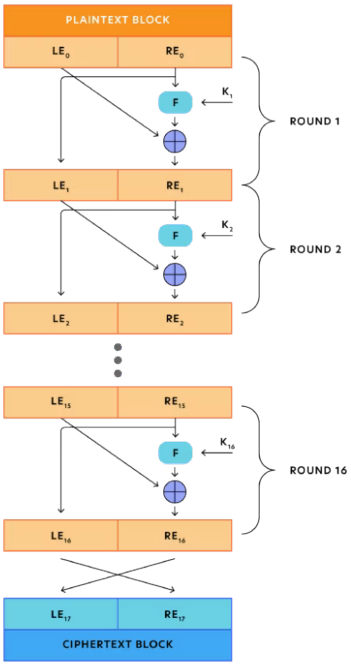
2.1 *Confusion* dan *Diffusion*

Menurut the GOAT Claude Shannon, dua konsep ini adalah syarat sebuah cipher yang baik. *Confusion* mengaburkan hubungan antara kunci dan teks sandi, sedangkan *diffusion* menyebarkan pengaruh setiap bit dari plainteks ke banyak bit cipherteks. Pada praktiknya, *confusion* dicapai dengan operasi nonlinier seperti S-box, sedangkan *diffusion* dicapai dengan operasi linier yang mencampur dan memindahkan bit, seperti permutasi. Kedua operasi tersebut diulang dalam beberapa putaran sehingga efeknya saling memperkuat.

2.2 Jaringan Feistel

Jaringan Feistel adalah struktur yang digunakan dalam banyak algoritma block cipher, termasuk DES. Struktur ini membagi blok input menjadi dua bagian, biasanya disebut sebagai *left* dan *right*. Setiap putaran dari jaringan Feistel melibatkan fungsi *round* yang diterapkan pada salah satu bagian, kemudian hasilnya digabungkan dengan bagian lainnya menggunakan operasi XOR. Proses ini diulang selama beberapa putaran, dan pada akhir proses, kedua bagian digabungkan kembali untuk membentuk blok output. Salah satu keuntungan utama dari jaringan Feistel adalah bahwa dekripsi dapat dilakukan dengan menggunakan algoritma yang sama dengan enkripsi, hanya dengan membalik urutan kunci putaran.

Keuntungan lain dari jaringan Feistel adalah fungsi *round* tidak perlu dapat dibalik, karena dekripsi cukup menghitung ulang fungsi tersebut lalu membatalkan operasi XOR. Apabila kedua bagian blok berukuran tidak sama, strukturnya disebut jaringan Feistel tak seimbang (*unbalanced Feistel network*). Selain XOR, penggabungan hasil fungsi *round* juga dapat menggunakan operasi lain yang memiliki invers, misalnya penjumlahan modulo 2^n, sehingga dekripsi menggunakan pengurangan modulo 2^n.



Gambar 4: Struktur Jaringan Feistel

2.3 Substitusi dan Permutasi

Substitusi adalah proses mengganti satu simbol dengan simbol lain, sedangkan permutasi adalah proses mengatur ulang urutan simbol-simbol tersebut. Keduanya merupakan operasi dasar dalam enkripsi dan digunakan untuk mencapai prinsip *confusion* dan *diffusion*. Kombinasi dari substitusi dan permutasi membentuk dasar dari banyak algoritma block cipher modern, termasuk AES. Dalam AES, blok data diolah melalui beberapa putaran yang melibatkan substitusi byte menggunakan S-box, pergeseran baris, pencampuran kolom, dan penambahan kunci putaran.

Beberapa operasi yang termasuk dalam substitusi dan permutasi adalah `SubBytes` yang mengganti setiap byte dengan byte lain menggunakan S-box, `ShiftRows` yang menggeser baris-baris dari blok data, `MixColumns` yang mencampur kolom-kolom dari blok data, dan `AddRoundKey` yang menambahkan kunci putaran ke blok data menggunakan operasi XOR.

S-box yang baik bersifat bijektif dan nonlinier. Salah satu cara membangunnya, seperti pada AES, adalah menghitung invers multiplikatif setiap byte di medan Galois $GF(2^8)$, yaitu himpunan polinomial berderajat kurang dari 8 dengan koefisien biner yang dioperasikan modulo sebuah polinomial tak tereduksi berderajat 8. Invers multiplikatif memberikan nonlinieritas yang tinggi, kemudian hasilnya dikenai transformasi afin untuk menghilangkan struktur aljabar yang terlalu sederhana. Tabel hasil konstruksi ini disimpan sehingga substitusi cukup dilakukan dengan *table lookup*.

2.4 Operasi Tambahan

Selain substitusi dan permutasi, *block cipher* umumnya menggunakan operasi tambahan yang sederhana dan dapat dibalik.

- **Rotasi bit**, yaitu pergeseran sirkular sebanyak k posisi. Inversnya adalah rotasi ke arah berlawanan.
- **Penjumlahan modulo** 2^n. Operasi ini nonlinier terhadap XOR sehingga menambah *confusion*, dan inversnya adalah pengurangan modulo 2^n.
- ***Xorshift***, yaitu operasi $x \leftarrow x \oplus (x \ll k)$ atau $x \leftarrow x \oplus (x \gg k)$ yang diperkenalkan oleh Marsaglia pada pembangkit bilangan acak. Operasi ini menyebarkan satu bit ke beberapa posisi dan dapat dibalik karena sebagian bit tidak berubah dan dapat dipulihkan secara bertahap.
- **Kompresi dan ekspansi bit**, yaitu pemetaan antara ukuran bit yang berbeda. Kompresi tidak dapat dibalik, sehingga hanya aman digunakan sebagai *mask* atau di dalam fungsi *round* Feistel.

2.5 Fungsi Kompresi SHA-256

SHA-256 adalah fungsi hash yang memproses pesan dalam blok 512 bit. Setiap blok diperluas menjadi 64 kata 32-bit W_t (*message schedule*), kemudian diolah oleh fungsi kompresi selama 64 putaran yang memperbarui delapan variabel kerja $a, b, \ldots, h$. Setiap putaran menggunakan rotasi, pergeseran, penjumlahan modulo 2^{32}, fungsi nonlinier Ch dan Maj, serta konstanta putaran K_t. Fungsi kompresi ini dirancang agar perubahan kecil pada masukan mengubah keluaran secara acak dan sulit dibalik, sehingga cocok digunakan sebagai fungsi *round* pada jaringan Feistel. Apabila konstanta K_t diturunkan dari kunci, fungsi ini berperan sebagai fungsi berkunci (*keyed function*).

2.6 Penjadwalan Kunci

Penjadwalan kunci (*key schedule*) membangkitkan kunci putaran yang berbeda untuk setiap putaran dari kunci utama. Konstanta putaran yang berbeda di setiap putaran diperlukan agar setiap putaran tidak identik, karena putaran yang identik membuat cipher rentan terhadap *slide attack*. Apabila satu putaran membutuhkan beberapa kunci, kunci-kunci tersebut dapat diturunkan dari kunci putaran yang sama menggunakan konstanta yang berbeda. Teknik ini disebut pemisahan domain (*domain separation*) dan mencegah dua komponen menggunakan kunci yang identik.

2.7 Mode Operasi dan *Padding*

Block cipher hanya mengenkripsi satu blok berukuran tetap, sehingga pesan yang lebih panjang diproses menggunakan mode operasi. Mode selain ECB menggunakan *Initialization Vector* (IV) atau *counter* agar plainteks yang sama menghasilkan cipherteks yang berbeda.

Tabel 1: Ringkasan mode operasi *block cipher*

Mode	**Enkripsi blok ke-i**	**Keterangan**
ECB	$C_i = E_K(P_i)$	Tanpa IV; blok plainteks yang sama menghasilkan blok cipherteks yang sama.
CBC	$C_i = E_K(P_i \oplus C_{i-1})$, $C_0 = IV$	Setiap blok bergantung pada blok sebelumnya.
CFB	$C_i = P_i \oplus E_K(C_{i-1})$, $C_0 = IV$	Bekerja seperti *stream cipher*; hanya membutuhkan E_K.
OFB	$O_i = E_K(O_{i-1})$, $O_0 = IV$; $C_i = P_i \oplus O_i$	*Keystream* tidak bergantung pada plainteks.
CTR	$C_i = P_i \oplus E_K(ctr + i)$	Setiap blok dapat diproses secara independen.

3 Perancangan Block Cipher

3.1 Desain Blok dan Kunci

OATSIDE menggunakan blok berukuran 64 bit sebagai konfigurasi utama, dan juga mendukung ukuran blok 96, 128, serta 192 bit. Panjang kunci utama sama dengan ukuran blok. Jumlah putaran bawaan adalah 16 dan dapat diubah melalui parameter program. Kecuali disebutkan lain, penjelasan pada bab ini menggunakan blok 64 bit.

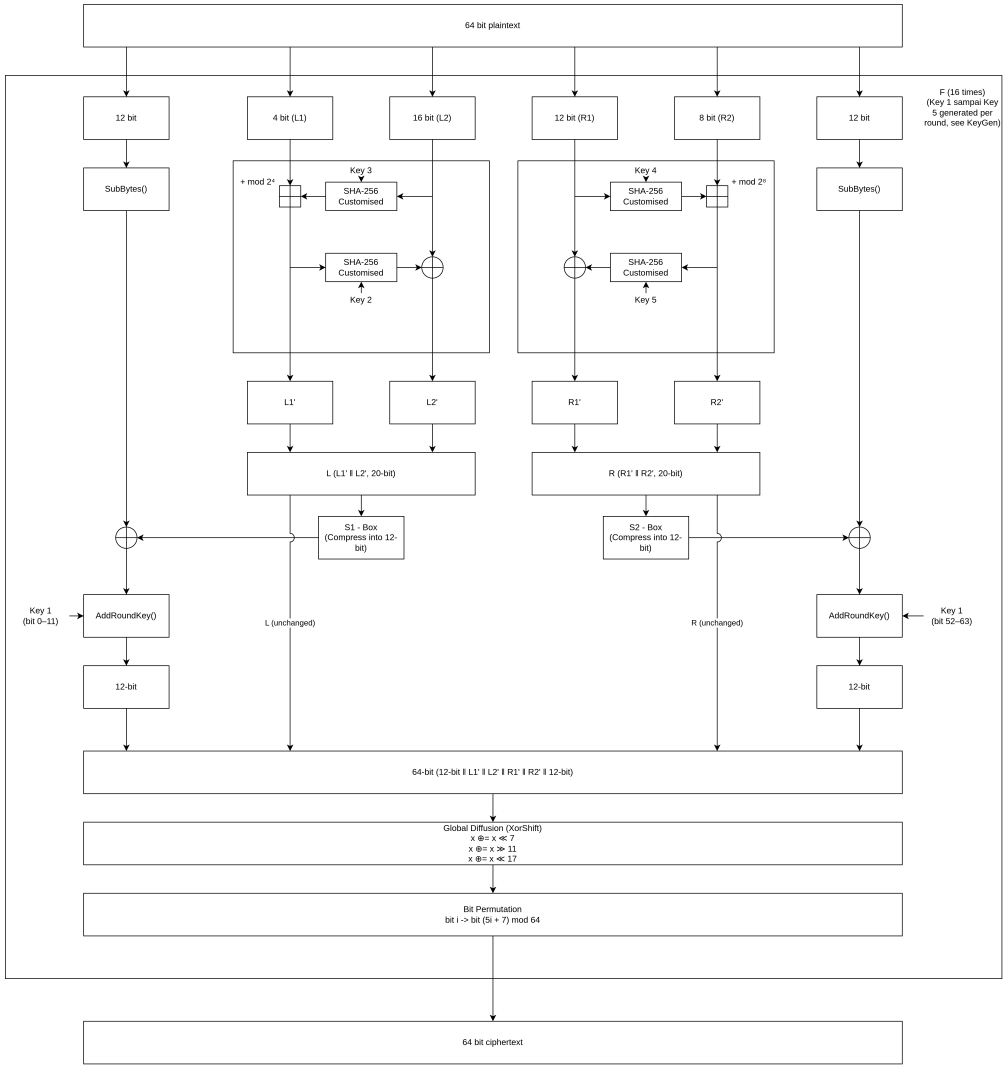
Pada setiap putaran, blok B bit dipecah menjadi enam cabang dengan urutan dari bit paling signifikan (MSB), yakni cabang luar kiri O_L, cabang dalam L_1, L_2, R_1, R_2, dan cabang luar kanan O_R. Lebar setiap cabang merupakan proporsi tetap dari B, seperti ditunjukkan pada tabel berikut. Cabang $L_1 \| L_2$ dan $R_1 \| R_2$ masing-masing berukuran $5B/16$ bit.

Tabel 2: Lebar cabang (bit) untuk setiap ukuran blok

Ukuran blok	O_L $(3B/16)$	L_1 $(B/16)$	L_2 $(4B/16)$	R_1 $(3B/16)$	R_2 $(2B/16)$	O_R $(3B/16)$
64	12	4	16	12	8	12
96	18	6	24	18	12	18
128	24	8	32	24	16	24
192	36	12	48	36	24	36

3.2 Struktur Putaran dan Enkripsi

Diagram satu putaran enkripsi ditunjukkan pada Gambar 5. Setiap putaran ke-r menggunakan lima subkunci $K_1, \ldots, K_5$ yang diturunkan dari kunci putaran ke-r (nanti kami bahas di Subbab 3.4 hehe). Notasi $H_K(x)$ menyatakan fungsi SHA-256 custom kami dengan subkunci K yang keluarannya dipotong ke lebar cabang tujuan, $C(\cdot)$ menyatakan kompresi 20 bit ke 12 bit, dan $S(\cdot)$ menyatakan substitusi S-box per byte.



Gambar 5: Diagram satu putaran enkripsi OATSIDE (blok 64 bit)

Satu putaran terdiri atas empat tahap berikut.

1. **Feistel kiri.** Pasangan (L_1, L_2) diperbarui dengan

$$L_1' = (L_1 + H_{K_3}(L_2)) \bmod 2^4, \qquad L_2' = L_2 \oplus H_{K_2}(L_1').$$

2. **Feistel kanan.** Pasangan (R_1, R_2) diperbarui dengan

$$R_2' = (R_2 + H_{K_4}(R_1)) \bmod 2^8, \qquad R_1' = R_1 \oplus H_{K_5}(R_2').$$

3. **Cabang luar.** Cabang luar disubstitusi, lalu di-XOR dengan hasil kompresi cabang dalam dan sebagian kunci K_1:

$$O_L' = S(O_L) \oplus C(L_1' \| L_2') \oplus K_1[0..11],$$
$$O_R' = S(O_R) \oplus C(R_1' \| R_2') \oplus K_1[52..63],$$

   dengan $K_1[a..b]$ menyatakan bit ke-a sampai ke-b dari K_1 yang dihitung dari MSB.
4. **Difusi global dan permutasi.** Keenam cabang digabung kembali menjadi state 64 bit x,

lalu diolah dengan tiga operasi *xorshift* dan permutasi bit:

$$x \leftarrow x \oplus (x \ll 7), \quad x \leftarrow x \oplus (x \gg 11), \quad x \leftarrow x \oplus (x \ll 17),$$

lalu permutasi P memindahkan bit ke-i ke posisi $(5i + 7) \bmod 64$.

Enkripsi satu blok menjalankan keempat tahap di atas sebanyak jumlah putaran, kemudian state terakhir menjadi blok cipherteks. Implementasi inti juga menerima nilai *whitening* awal yang di-XOR ke plainteks, tetapi seluruh mode operasi selalu mengisinya dengan nol. IV dan *counter* ditangani pada tingkat mode operasi, bukan di dalam fungsi enkripsi blok.

3.3 Komponen Algoritma

3.3.1 S-box

S-box OATSIDE berukuran 8 bit dan dibangkitkan satu kali saat program dimulai. Untuk setiap byte b, dihitung b^{-1} di $GF(2^8)$ dengan polinomial $x^8 + x^4 + x^3 + x^2 + 1$ (`0x11D`), dengan ketentuan $0^{-1} = 0$. Hasilnya dikenai transformasi *affine*[1]

$$S(b) = b^{-1} \oplus \mathrm{ROTL}_2(b^{-1}) \oplus \mathrm{ROTL}_5(b^{-1}) \oplus \texttt{0xA7},$$

dengan ROTL_k menyatakan rotasi kiri sebanyak k bit. Ini sebenarnya terinspirasi dari AES, tetapi polinomial, rotasi, dan konstanta *affine*-nya berbeda sehingga menghasilkan tabel yang berbeda.

Tabel 3: S-box OATSIDE

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**A**	**B**	**C**	**D**	**E**	**F**
0	A7	82	C2	1E	15	6A	7B	A0	09	FF	B6	4E	C9	F2	24	B9
1	07	52	7C	95	D8	A6	53	94	67	D9	0D	35	91	5A	A8	60
2	F7	40	5D	99	4A	62	BE	56	18	2E	27	8D	DD	73	3E	BD
3	C7	39	98	F4	05	13	EE	4B	BC	49	AE	9D	D7	1A	44	FD
4	8F	6E	A3	9F	2D	19	4F	33	51	1C	45	02	2B	B1	5F	08
5	78	6C	63	88	10	96	B2	6F	9A	59	3A	25	9C	50	AA	CB
6	97	47	1F	FA	CF	E8	0E	76	F6	AD	0A	AC	03	65	D1	1B
7	2A	0B	D0	22	D4	CC	4D	14	68	E3	79	85	A1	E4	7D	37
8	B3	28	B4	6D	A5	C1	4C	C3	E2	74	F8	9B	D3	46	ED	3B
9	DC	FC	7A	3C	21	7E	75	01	E1	93	5B	F3	2C	AB	87	DB
A	48	7F	B5	A2	C5	36	30	16	8B	17	3F	EC	DA	8E	34	38
B	CE	69	2F	0C	9E	C0	11	43	CD	DE	5C	5E	D6	04	66	1D
C	BF	B0	20	E0	FB	EA	FE	90	64	71	00	DF	84	80	B8	32
D	0F	3D	55	54	86	81	D5	F0	F5	77	C6	31	6B	C4	F9	42
E	61	83	06	8A	EB	E6	92	AF	E9	8C	E5	57	D2	26	89	29
F	B7	23	72	BB	C8	E7	41	70	A4	BA	F1	A9	CA	58	EF	12

3.3.2 Kompresi Cabang Dalam

Setelah semua tahap tadi selesai, kompresi C memetakan 20 bit $L_1' \| L_2'$ atau $R_1' \| R_2'$ menjadi 12 bit. Data 20 bit terlebih dahulu disubstitusi per byte, sehingga dua byte bawah melalui

[1] kurang tahu padanan bahasa Indonesianya apa hehe.

S-box dan 4 bit teratas diteruskan. Hasil s kemudian dilipat menjadi

$$C(v) = (s \gg 8) \oplus ((s \bmod 2^8) \ll 3),$$

yaitu 12 bit teratas di-XOR dengan 8 bit terbawah yang digeser 3 posisi.

3.3.3 Difusi Global dan Permutasi Bit

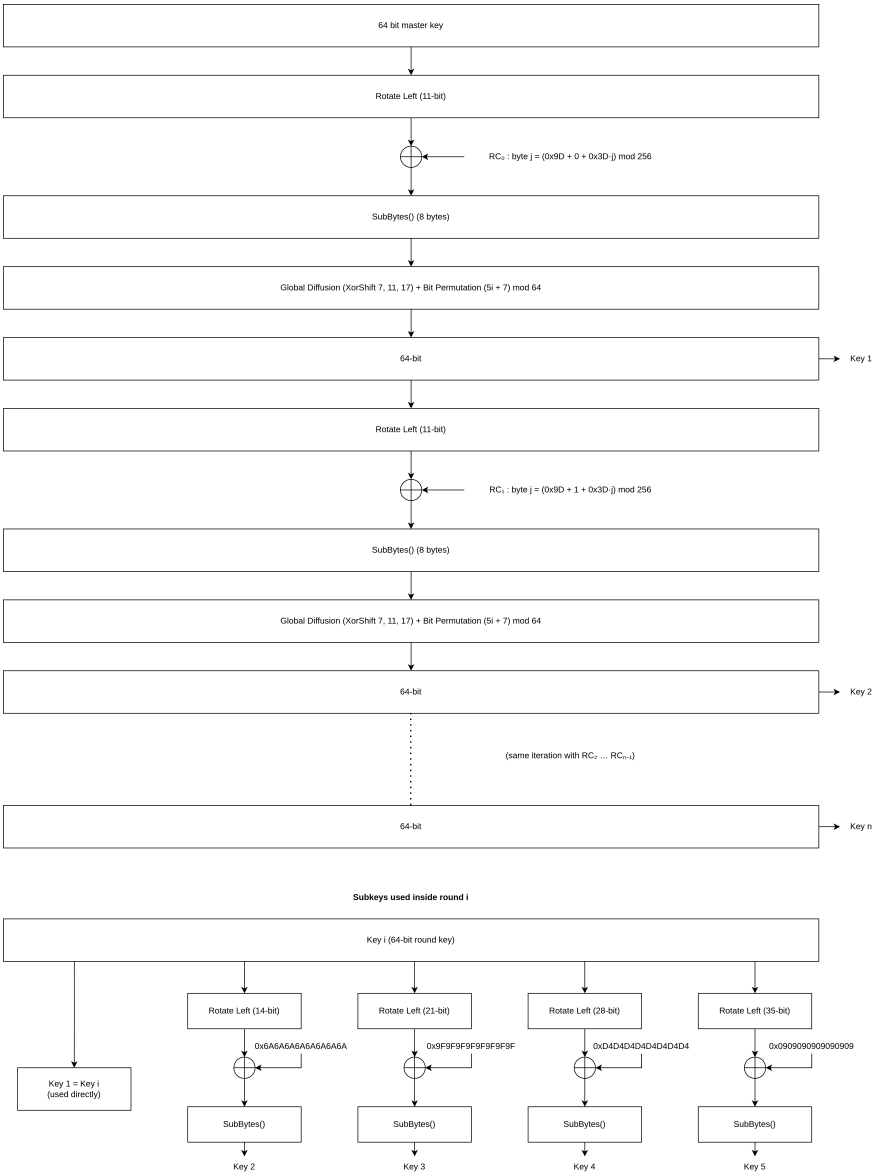
Di dalam satu putaran, cabang kiri dan kanan belum saling memengaruhi, dan cabang luar belum memengaruhi cabang dalam. Tahap difusi global mengatasi hal tersebut. Tiga operasi *xorshift* menyebarkan setiap bit ke beberapa posisi lain, yaitu 2 sampai 8 bit keluaran untuk setiap bit masukan pada blok 64 bit. Permutasi P kemudian memindahkan bit ke-i (dihitung dari LSB) ke posisi $(5i + 7) \bmod B$. Karena $\gcd(5, B) = 1$ untuk seluruh ukuran blok yang didukung (permutasi ini bijektif btw).

3.4 Penjadwalan Kunci

Diagram penjadwalan kunci ditunjukkan pada Gambar 6. Kunci putaran dibangkitkan secara berantai dari kunci utama M. Dengan $s_0 = M$, untuk setiap $i = 0, \ldots, n-1$ dihitung

$$s_{i+1} = D(S(\mathrm{ROTL}_{11}(s_i) \oplus RC_i)), \qquad RK_{i+1} = s_{i+1},$$

dengan D adalah difusi global dan permutasi bit yang sama dengan pada putaran enkripsi. Konstanta putaran RC_i disusun dari byte-byte $(\texttt{0x9D} + i + \texttt{0x3D} \cdot j) \bmod 256$ untuk $j = 0, 1, \ldots$ dari MSB, sehingga setiap putaran memiliki konstanta yang berbeda.



Gambar 6: Diagram penjadwalan kunci dan penurunan subkunci OATSIDE

Dari setiap kunci putaran RK diturunkan lima subkunci menggunakan pemisahan domain. Subkunci K_1 adalah RK itu sendiri, sedangkan untuk $d = 2, 3, 4, 5$ berlaku

$$K_d = S\big(\mathrm{ROTL}_{7d}(RK) \oplus D_d\big),$$

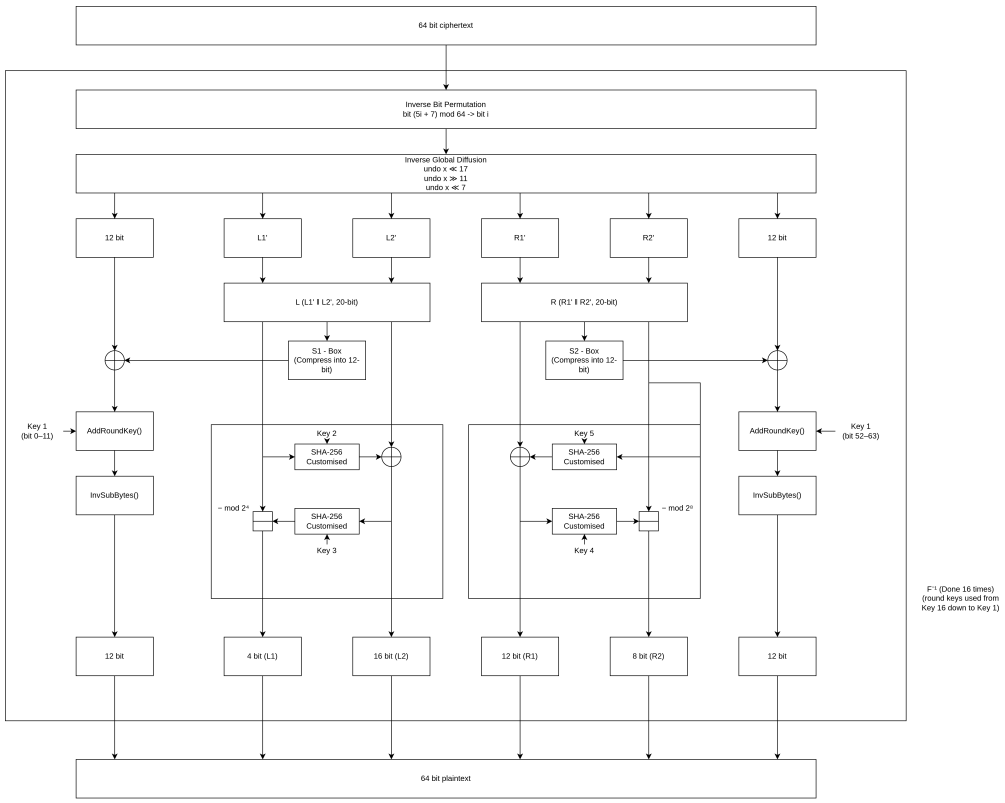
dengan D_d adalah byte $(\texttt{0x35} \cdot d)$ mod 256 yang diulang sepanjang blok. Rincian subkunci dan pemakaiannya ditunjukkan pada Tabel 4.

Tabel 4: Subkunci dalam satu putaran (blok 64 bit)

Subkunci	**Rotasi**	**Konstanta**	**Digunakan pada**
K_1	–	–	XOR cabang luar (O_L dan O_R)
K_2	14	`0x6A6A...6A`	H kedua Feistel kiri
K_3	21	`0x9F9F...9F`	H pertama Feistel kiri
K_4	28	`0xD4D4...D4`	H pertama Feistel kanan
K_5	35	`0x0909...09`	H kedua Feistel kanan

3.5 Dekripsi

Diagram dekripsi ditunjukkan pada Gambar 7. *Basically* proses dekripsi adalah kebalikan dari enkripsi, bahkan urutan subkuncinya juga terbalik.



Gambar 7: Diagram satu putaran dekripsi OATSIDE (blok 64 bit)

4 Implementasi dan Simulasi

Implementasi utama OATSIDE ditulis dalam bahasa Python 3.10, tetapi putaran kompresi SHA-256 Customised juga diimplementasikan dalam bahasa C untuk meningkatkan performa[2]. Pustaka hasil kompilasi C bersifat opsional. Apabila pustaka tersebut tidak tersedia, program secara otomatis menggunakan implementasi Python. Seluruh cipher, mode operasi, dan *padding* diimplementasikan sendiri tanpa pustaka kriptografi pihak ketiga.

4.1 Struktur

Struktur direktori proyek dapat dilihat pada Tabel 5.

Tabel 5: Struktur direktori proyek

Direktori/Berkas	**Isi**
`main.py`	Titik masuk program (CLI).
`src/`	Implementasi OATSIDE: inti cipher (`cipher_core.py`), S-box dan SHA-256 Customised (`sbox_spn.py`), penjadwalan kunci (`keygen.py`), operasi bit (`bit_utils.py`), mode operasi dan *padding* (`modes.py`), CLI (`cli.py`), serta fungsi analisis (`analysis.py`).
`src/native/`	Kode sumber C untuk putaran kompresi SHA-256 (lebih cepet dri Python).
`config/`	Parameter lebar cabang untuk setiap ukuran blok.
`analysis/`	Skrip eksperimen *avalanche effect*, entropi, dan histogram.
`tests/`	Pengujian unit, batas ukuran data, konstruksi mode, dan integrasi CLI.
`docs/`	Diagram rancangan, tabel S-box, dokumentasi API, dan laporan.

4.2 Antarmuka Program

Tabel 6: Parameter CLI

Parameter	**Keterangan**
`-i`, `-input`	Path berkas masukan
`-o`, `-output`	Path berkas keluaran; harus berbeda dari berkas masukan
`-k`, `-key`	Kunci utama dalam heksadesimal sepanjang ukuran blok (16 digit untuk blok 64 bit)
`-m`, `-mode`	ECB, CBC, CFB, OFB, atau CTR (bawaan: CBC)
`-b`, `-block-bits`	Ukuran blok 64, 96, 128, atau 192 bit (bawaan: 64)
`-iv`	IV untuk CBC, CFB, dan OFB, atau *counter* awal untuk CTR, dalam heksadesimal sepanjang satu blok
`-r`, `-rounds`	Jumlah putaran (bawaan: 16)

Contoh enkripsi dan dekripsi berkas dengan mode CBC begini:

[2]Awalnya kami menggunakan sepenuhnya pakaii Python, tetapi *overhead* interpreter Python membuat fungsi ini menjadi bagian paling lambat dari cipher. Versi C terdapat pada `src/native/oatside_sha.c` dan menghasilkan keluaran yang identik dengan versi Python.

```
python main.py encrypt -i image.png -o image.enc -k 0x1234567890ABCDEF
    -m CBC --iv 0x0011223344556677
python main.py decrypt -i image.enc -o image.dec.png -k 0
   x1234567890ABCDEF -m CBC --iv 0x0011223344556677
```

4.3 Hasil Uji Coba

Uji coba dilakukan melalui CLI pada Linux (kernel 6.12) dengan Python 3.13 dan pustaka C yang dikompilasi menggunakan GCC 14.2. Kunci yang digunakan adalah `0x1234567890ABCDEF`, IV/*counter* adalah `0x0011223344556677`, ukuran blok 64 bit, dan 16 putaran. Setiap berkas dienkripsi lalu didekripsi dengan kelima mode, kemudian nilai SHA-256 berkas hasil dekripsi dibandingkan dengan berkas asli. Hasilnya ditunjukkan pada Tabel 7.

Tabel 7: Hasil uji coba enkripsi dan dekripsi berkas (blok 64 bit)

Berkas	**Plainteks (byte)**	**Cipherteks ECB/CBC**	**Cipherteks CFB/OFB/CTR**	**SHA-256 asli = hasil dekripsi (16 digit awal)**
Teks (`.txt`)	47	48	47	`74cc52e71fb3a81a`
Berkas kosong	0	8	0	`e3b0c44298fc1c14`
Biner 1 byte	1	8	1	`559aead08264d579`
Biner 8 byte	8	16	8	`6c03f3ef0ba21f4b`
Byte `00`–`FF` berulang	1.224	1.232	1.224	`7c94ffe7ba39666b`
Arsip ZIP	1.108	1.112	1.108	`ec7fdf0c908029c4`
Potongan PDF	30.000	30.008	30.000	`890f34e9a9bfbdfa`
Gambar PNG	139.719	139.720	139.719	`b8132e40693cc865`

Seluruh 40 kombinasi berkas dan mode menghasilkan berkas dekripsi yang identik byte demi byte dengan berkas asli. Berkas *byte* `00`–`FF` berisi seluruh nilai byte dan deretan panjang byte `00` serta `FF` untuk memastikan program tidak mengasumsikan teks UTF-8. Pengujian yang sama untuk berkas ZIP dengan ukuran blok 96, 128, dan 192 bit pada kelima mode juga menghasilkan berkas yang identik. Selain itu, rangkaian pengujian otomatis (`python -m unittest discover -s tests`) yang berisi 42 kasus uji, termasuk uji konstruksi mode yang dihitung terpisah dari implementasi mode, lulus seluruhnya.

Waktu enkripsi melalui CLI untuk gambar PNG berukuran 139.719 byte adalah sekitar 11,5–12,2 detik pada setiap mode, atau sekitar 11–12 KB/s. Waktu dekripsi hampir sama. Kecepatan ini dipengaruhi oleh rancangan OATSIDE yang memanggil 64 kompresi SHA-256 untuk setiap blok.

4.4 Verifikasi Integritas

Integritas pada program ini diartikan sebagai jaminan bahwa hasil dekripsi identik dengan data asli apabila kunci, mode, dan IV yang digunakan sama. Ini dibuktikan melalui perbandingan nilai SHA-256 pada Tabel 7. Ada bberapa kondisi kesalahan juga diuji sebagai berikut.

- Dekripsi ECB dengan kunci yang salah satu bitnya berbeda ditolak dengan pesan *padding* PKCS#7 tidak valid.

- Perubahan satu bit pada byte terakhir cipherteks CBC ditolak dengan pesan *padding* tidak valid.
- Cipherteks CBC yang panjangnya bukan kelipatan ukuran blok ditolak sebelum dekripsi dilakukan.
- Perubahan satu bit pada cipherteks CTR tetap didekripsi tanpa pesan kesalahan, dan menghasilkan satu bit yang berbeda pada posisi yang sama di plainteks.

Nah, kasus terakhir menunjukkan bahwa program tidak menyediakan autentikasi cipherteks. Penolakan karena *padding* tidak valid hanya terjadi secara kebetulan pada ECB dan CBC, sehingga tidak dapat dianggap sebagai mekanisme pendeteksian manipulasi.

4.5 Analisis

Hasil uji coba menunjukkan bahwa kelima mode operasi berjalan dengan benar untuk file teks dan biner, termasuk file kosong dan file yang ukurannya tepat di batas blok. Perbedaan panjang cipherteks sesuai dengan rancangan, yaitu ECB dan CBC menambahkan 1 sampai 8 byte *padding*, sedangkan CFB, OFB, dan CTR menghasilkan cipherteks dengan panjang yang sama dengan plainteks. Mode CFB, OFB, dan CTR lebih cocok untuk data yang panjangnya tidak boleh berubah, sedangkan ECB tidak disarankan untuk data dengan pola berulang karena blok plainteks yang sama menghasilkan blok cipherteks yang sama. Keterbatasan utama program adalah kecepatan, yaitu sekitar 11–12 KB/s, sehingga program lebih sesuai untuk file berukuran kecil hingga sedang.

5 Analisis Keamanan

Seluruh pengujian pada bab ini menggunakan blok 64 bit dan 16 putaran, serta dijalankan dengan skrip eksperimen pada repositori menggunakan perintah berikut.

```
python3 -m analysis.run_experiments --trials 256 --sample-bytes 4096
    --seed 4020 --plots
```

Opsi `-plots` membutuhkan pustaka `matplotlib`. Skrip menyimpan *seed*, kunci, IV, data mentah setiap percobaan (`avalanche_trials.csv`), frekuensi byte (`histograms.csv`), ringkasan (`results.json`), dan grafik histogram setiap mode pada direktori `analysis/results/`, sehingga hasilnya dapat direproduksi.

5.1 *Avalanche Effect*

Pada setiap percobaan, dibangkitkan plainteks satu blok, kunci, dan IV secara acak. Kemudian satu bit acak pada plainteks atau pada kunci dibalik, sedangkan masukan lainnya tetap. Persentase *avalanche* dihitung sebagai jumlah bit cipherteks yang berbeda dibagi 64 bit. Blok *padding* PKCS#7 yang ditambahkan oleh ECB dan CBC tidak ikut dibandingkan, karena blok tersebut tidak bergantung pada bit plainteks yang dibalik dan akan menurunkan persentase secara semu. Hasil 256 percobaan untuk setiap mode ditunjukkan pada Tabel 8.

Tabel 8: Hasil uji *avalanche effect* (256 percobaan, persentase dari 64 bit)

	1 bit plainteks dibalik				**1 bit kunci dibalik**			
Mode	**Rata-rata**	**Min**	**Maks**	**Simp. baku**	**Rata-rata**	**Min**	**Maks**	**Simp. baku**
ECB	50,22	34,38	70,31	6,51	50,74	32,81	67,19	6,17
CBC	50,21	31,25	67,19	6,20	49,84	34,38	65,62	6,13
CFB	1,56	1,56	1,56	0,00	49,54	35,94	67,19	6,12
OFB	1,56	1,56	1,56	0,00	49,54	35,94	67,19	6,12
CTR	1,56	1,56	1,56	0,00	49,54	35,94	67,19	6,12

Pada ECB dan CBC, perubahan satu bit plainteks mengubah rata-rata sekitar 50% bit cipherteks, sesuai dengan nilai ideal. Simpangan baku sekitar 6,2% juga mendekati nilai yang diharapkan apabila setiap bit keluaran berubah secara acak dengan peluang 0,5, yaitu $\sqrt{64 \times 0{,}25}/64 = 6{,}25\%$. Perubahan satu bit kunci menghasilkan sekitar 50% bit berubah pada kelima mode.

Pada CFB, OFB, dan CTR, perubahan satu bit plainteks hanya mengubah tepat satu bit cipherteks ($1{,}56\% = 1/64$). Hal ini merupakan sifat konstruksi mode, bukan kelemahan cipher, karena pada ketiga mode tersebut plainteks hanya di-XOR dengan *keystream*. Nilai uji kunci ketiga mode juga identik karena untuk blok pertama ketiganya menghasilkan *keystream* yang sama, yaitu $E_K(IV)$.

5.2 Entropi

Entropi Shannon dihitung dengan $H = -\sum_x p(x) \log_2 p(x)$ untuk setiap nilai byte x, dengan nilai maksimum 8 bit/byte. Plainteks yang digunakan adalah sampel 4.096 byte berisi byte `00` sampai `07` yang diulang, sehingga entropinya tepat 3 bit/byte. Kunci dan IV yang digunakan dibangkitkan dari *seed* 4020, yaitu `0x34B334ACC96F4286` dan `0xE6C375B580FD1337`. Hasilnya ditunjukkan pada Tabel 9.

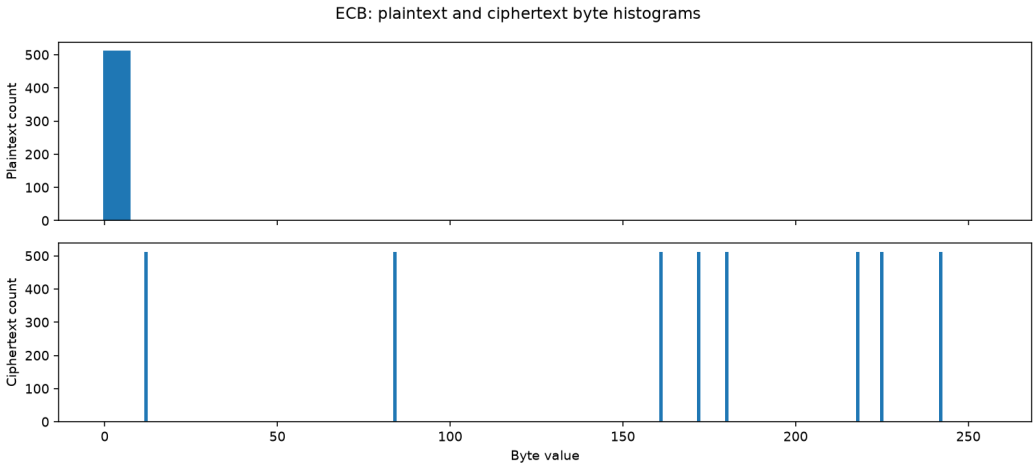
Tabel 9: Entropi plainteks dan cipherteks

Data	**Ukuran (byte)**	**Entropi (bit/byte)**
Plainteks	4.096	3,000
Cipherteks ECB	4.104	3,020
Cipherteks CBC	4.104	7,952
Cipherteks CFB	4.096	7,952
Cipherteks OFB	4.096	7,949
Cipherteks CTR	4.096	7,958

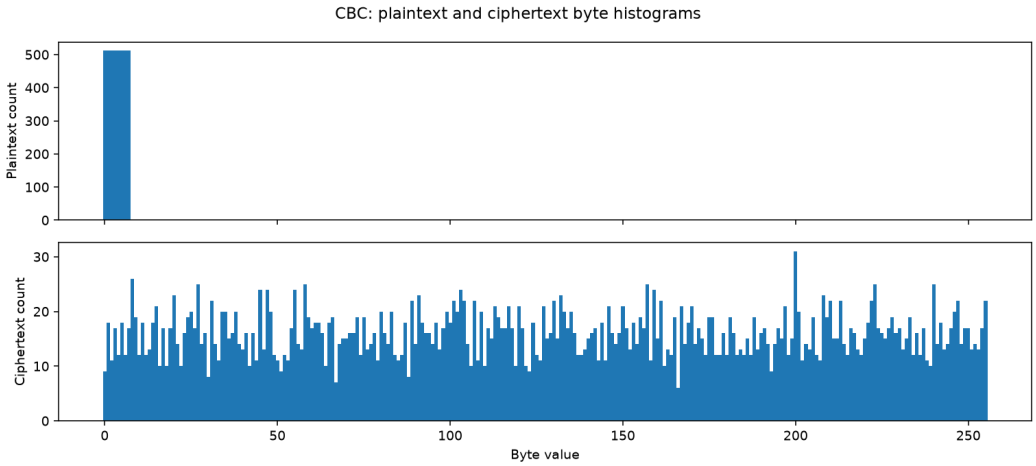
CBC, CFB, OFB, dan CTR menghasilkan entropi sekitar 7,95 bit/byte. Nilai ini tidak mencapai 8 karena jumlah sampel terbatas. Untuk data acak seragam berukuran sekitar 4.096 byte, nilai harapan entropinya adalah sekitar $8 - 255/(2n \ln 2) \approx 7{,}955$ bit/byte, sehingga hasil keempat mode tersebut setara dengan data acak. Sebaliknya, ECB hanya menghasilkan 3,02 bit/byte. Karena plainteks terdiri atas blok `00 01 ... 07` yang berulang, seluruh blok cipherteks ECB juga identik. Hal ini menunjukkan kelemahan ECB dalam menyembunyikan pola, bukan kelemahan cipher.

5.3 Histogram

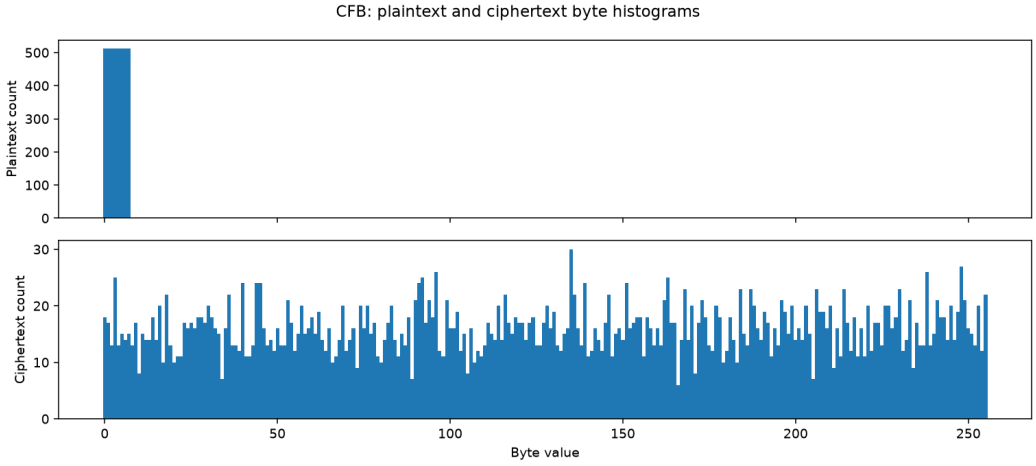
Gambar 8 sampai Gambar 12 membandingkan frekuensi kemunculan setiap nilai byte pada plainteks (panel atas) dan cipherteks (panel bawah) untuk setiap mode, menggunakan sampel yang sama dengan pengujian entropi.



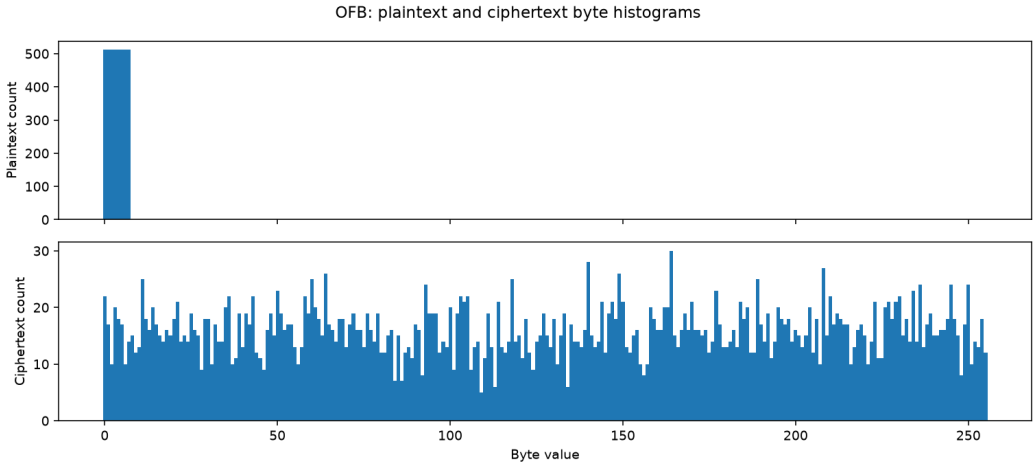
Gambar 8: Histogram byte plainteks dan cipherteks mode ECB



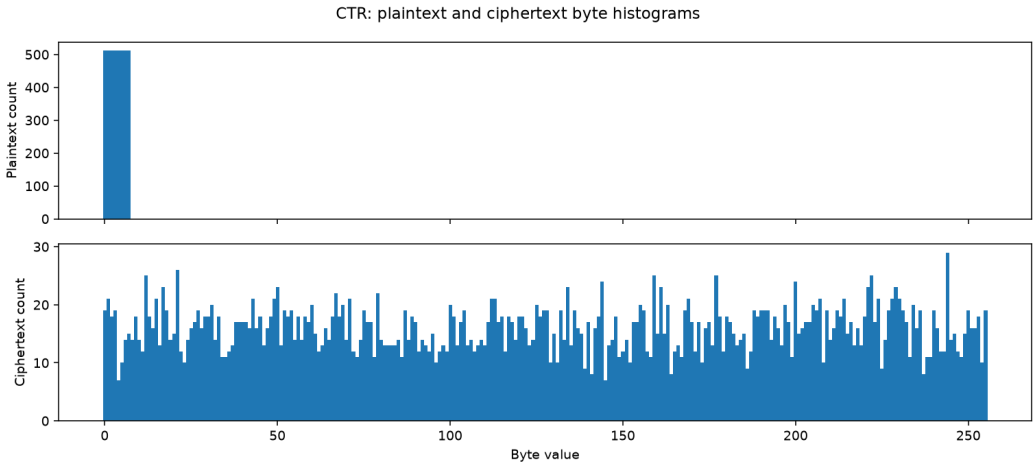
Gambar 9: Histogram byte plainteks dan cipherteks mode CBC



Gambar 10: Histogram byte plainteks dan cipherteks mode CFB



Gambar 11: Histogram byte plainteks dan cipherteks mode OFB



Gambar 12: Histogram byte plainteks dan cipherteks mode CTR

Plainteks hanya berisi delapan nilai byte, masing-masing muncul 512 kali. Pada CBC, CFB, OFB, dan CTR, cipherteks tersebar di seluruh rentang 0–255 dengan frekuensi sekitar $4.096/256 = 16$ untuk setiap nilai byte, sehingga distribusinya mendekati seragam dan pola plainteks tidak lagi terlihat. Pada ECB, cipherteks hanya berisi delapan nilai byte dengan frekuensi masing-masing sekitar 512, karena setiap blok plainteks yang identik dienkripsi menjadi blok cipherteks yang identik. Histogram ECB tersebut memperlihatkan secara langsung bahwa ECB membocorkan pola plainteks.

5.4 Keterbatasan Analisis

Hasil *avalanche effect* yang mendekati 50%, entropi yang setara dengan data acak, dan histogram yang mendekati seragam pada mode selain ECB menunjukkan bahwa OATSIDE tidak memiliki kelemahan statistik yang terlihat pada sampel yang diuji. Namun, hasil ini merupakan pengukuran empiris dan tidak membuktikan bahwa OATSIDE aman. Menurutt kami, ada banyak analisis lain yang bisa kita lakukan supaya bisa lebih yakin bahwa OATSIDE aman, misalnya analisis diferensial dan analisis linear.

6 Penutup

6.1 Kesimpulan

Dengan tugas ini, kami berhasil merancang dan mengimplementasikan algoritma block cipher baru bernama OATSIDE. Algoritma ini menggabungkan jaringan Feistel dan lapisan substitusi-permutasi dalam satu putaran, serta menambahkan mekanisme kompresi dan difusi global untuk meningkatkan keamanan. Implementasi OATSIDE dilakukan dalam bahasa Python 3.10, dengan beberapa bagian dioptimalkan menggunakan bahasa C untuk meningkatkan performa.

Berdasarkan hasil pengujian, OATSIDE menunjukkan performa yang baik dalam hal *avalanche effect*, entropi, dan distribusi histogram pada berbagai mode operasi enkripsi. Namun, kami menyadari bahwa masih ada ruang untuk perbaikan dan optimasi lebih lanjut, terutama dalam hal performa dan keamanan algoritma.

6.2 Saran

Sebaiknya pengembangan selanjutnya pakai bahasa yang lebih sigma seperti C atau Rust, jangan pakai Python karena lemot. Selain itu, kami punya hipotesis bahwa ada beberapa hal dari skema *block cipher*-nya yang *by design* kurang optimal secara performa maupun keamanan, jadi sebaiknya dicoba untuk diubah-ubah dan diuji lagi.

6.3 Lampiran

Tautan dokumentasi API: https://brianhadianstei23.github.io/13523048-13523094-135230115-tubes2-IF4020/
Video demo: